

Data Analytics (PETR 6397)

Dimensionality Reduction

Dr. Ahmad Sakhaee-Pour

Petroleum Engineering Department

University of Houston

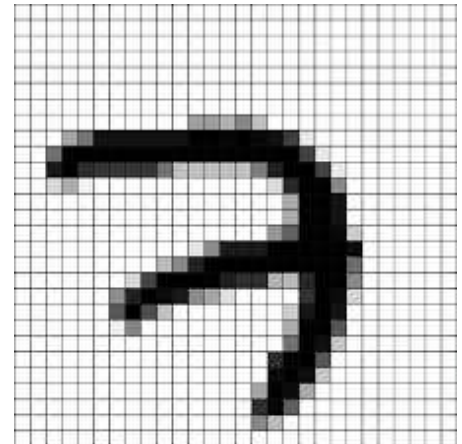
Spring 2025

Discussed topics

- Curse of dimensionality
- Dimensionality reduction
- Principal Component Analysis (PCA)
- Stochastic Neighbor learning (SNE)
- T-Distributed Stochastic Neighbor Embedding (t-SNE)

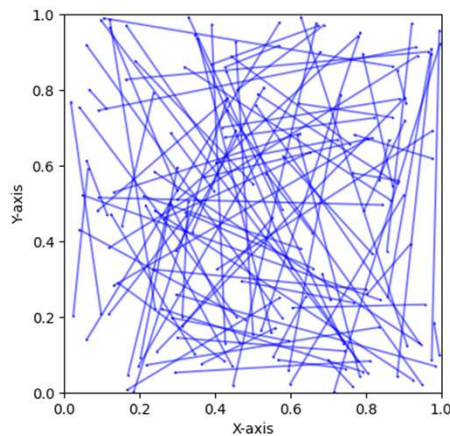
What is dimensionality

- Dimensionality is the number of dimensions (features) of a sample data
- What is the dimensionality of the MNIST image?
- What was the dimensionality of each rock sample in the previous homework?

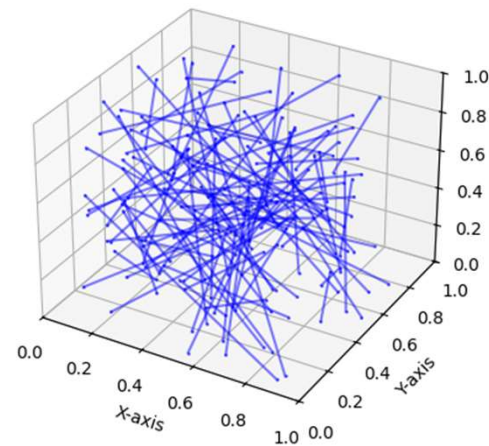


Placing two points randomly in a unit cell

- Suppose we have a unit cell where we place two points randomly
- Determine the distance between the points
- Repeat this process and average the results



100 trials in 2d



100 trials in 3d

Average distance between the two points increases in higher dimensions

- We placed 10,000 points and calculated the average distance
 - 1d: 0.3364
 - 2d: 0.5203
 - 3d: 0.6657
 - 4d: 0.7760
 - 10d: 1.2667
 - 100d: 4.0734
 - 1000d: 12.9036

Curse of dimensionality

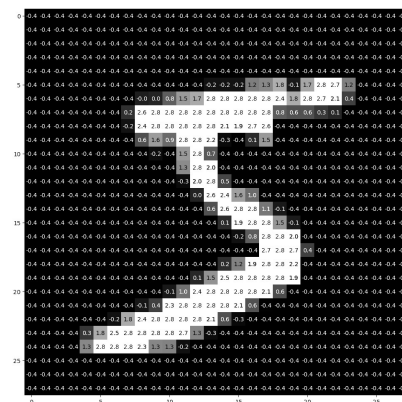
- Curse of dimensionality is a set of problems concerned with data in a higher-dimensional space:
 - A. Data sparsity happens because data become sparser
 - B. Visualization is more challenging
 - C. Distance is not as informative as 2d or 3d
 - ...

Data becomes sparser in higher dimensions

- Because of data sparsity, making valid correlations becomes more difficult
- We like to keep the useful information but minimize the negative effects of sparsity. This is the objective of dimensionality reduction

Benefits of the dimensionality reduction

- It helps the visualization by decreasing the number of dimensions (consider plotting your data in 2d or 3d versus 784d= 28x28 pixels)
- It may filter noise (many pixels are not important in MNIST data)
- It accelerates the process but does not necessarily improve performance because some information is removed
- Caution: Accelerating the training for low-dimensional data does not justify using the dimensionality reduction



Two applications of dimensionality reduction

1. Preprocessing and feature engineering: One of the standard methods of dimensionality reductions is the Principal Component Analysis (PCA), which extracts more important components
2. Generating plausible artificial datasets: PCA divides the dataset into components. We can then use this information to generate new samples (see the bonus homework problem)

There are two main approaches for data reduction

- Projection: projects data from a higher-dimensional space to a lower-dimensional subspace to remove the sparsity
- Manifold learning: data exist near a much lower-dimensional manifold that is difficult to detect in a high-dimensional space
- Basically, we try to find the actual degree of freedom:
 - What is the degree of freedom of a straight line when it is not horizontal or vertical?
 - How about a non-curved plane?

Applicability depends on the dataset: Compare projection vs manifold learning

Swiss roll dataset

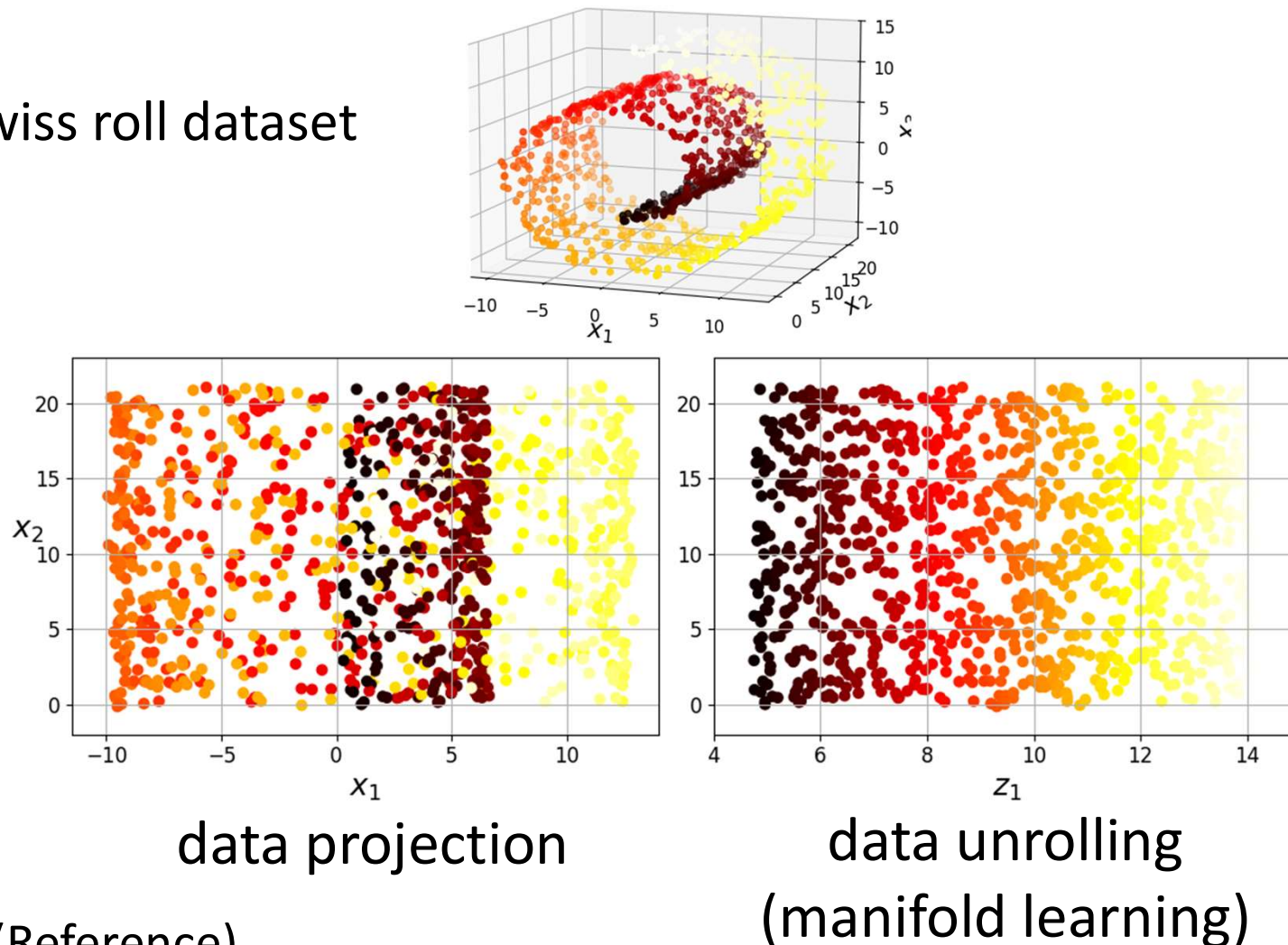


Fig. 8.5 (Reference)

Decision boundaries may become simpler or more complex depending on the data structure

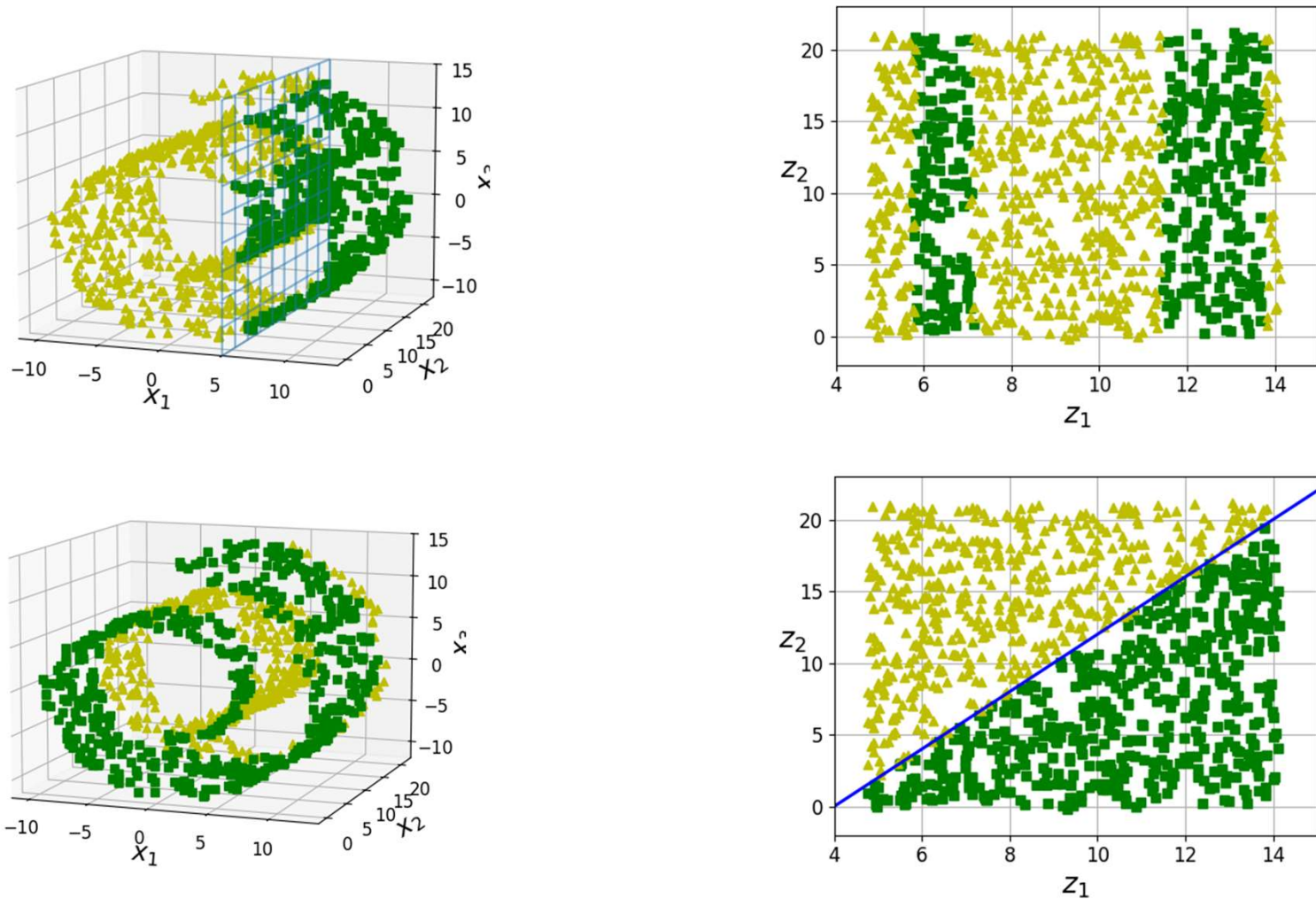
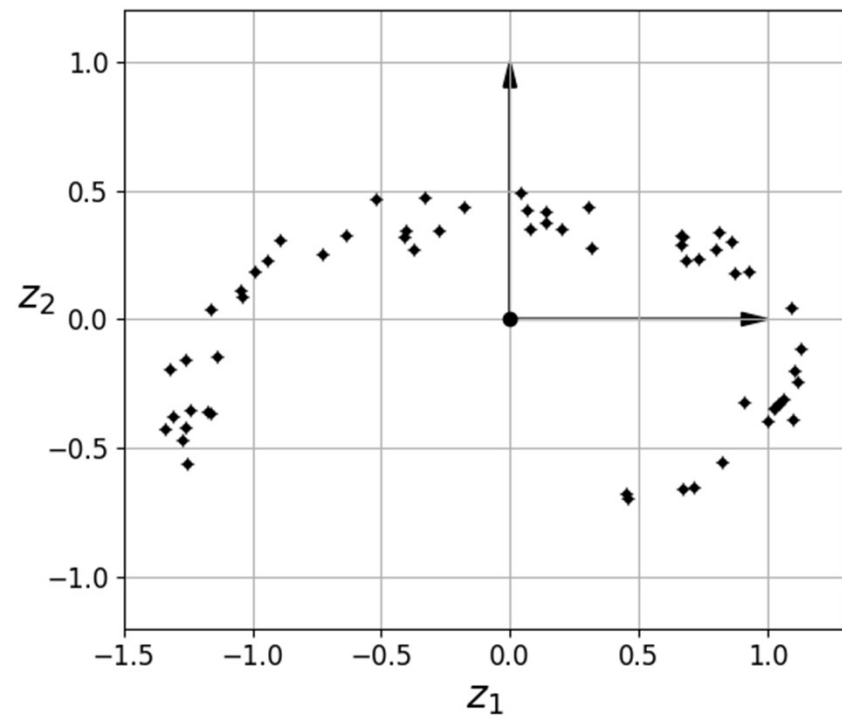
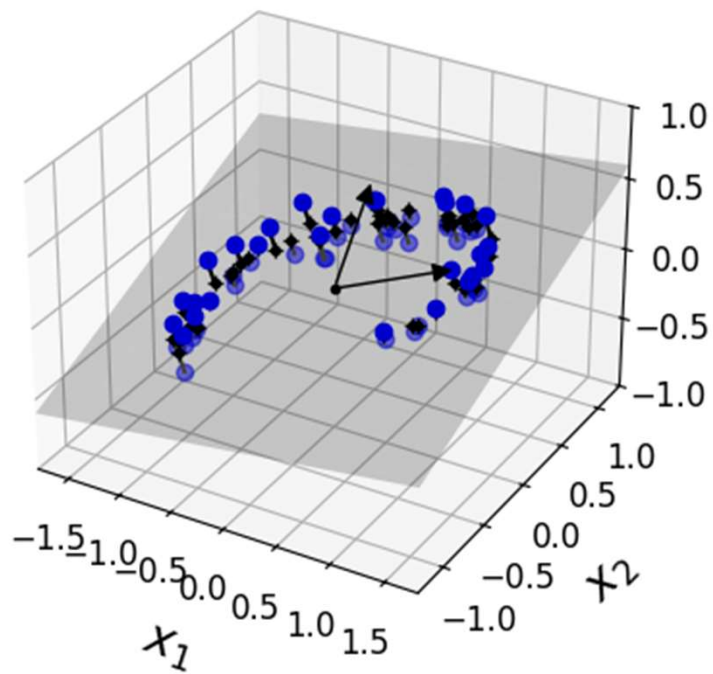


Fig. 8.6 (Reference)

Projection



Eigenvalue and eigenvector (reminder)

- Principal Component Analysis (PCA) is based on eigenvalue and eigenvector
- Eigenvalue (λ) and eigenvector (X) of a square matrix (A) are obtained from $A \cdot X = \lambda \cdot X$
- We first solve $\det(A - \lambda \cdot I) = 0$ to find the eigenvalues
- We then find the eigenvector for each eigenvalue
- Larger eigenvalues are more important and carry more information

Example of eigenvalue and eigenvector calculation

```
A = np.array([[3, 1, 4],  
              [1, 5, 9],  
              [2, 6, 5]])
```

Eigenvalues: [13.08576474 2.58000566 -2.66577041]

Eigenvectors: [[-0.31542644 -0.95117074 -0.32372474]
 [-0.72306109 0.30781323 -0.70222933]
 [-0.61456393 0.02291827 0.63409484]]

Note: eigenvectors are normal to each other

Q: Which eigenvalue is the least important?

PCA determines eigenvalues and eigenvectors using SVD

- Singular value decomposition (SVD)

$$A = U \times S \times V^T$$

- U contains the left singular vectors (columns of U)
- S is a diagonal matrix containing the singular values
- V contains the right singular vectors (columns of V)
- V^T : Is the transpose of V
- SVD applies to non-square matrix
- Its answer is different from the previous method

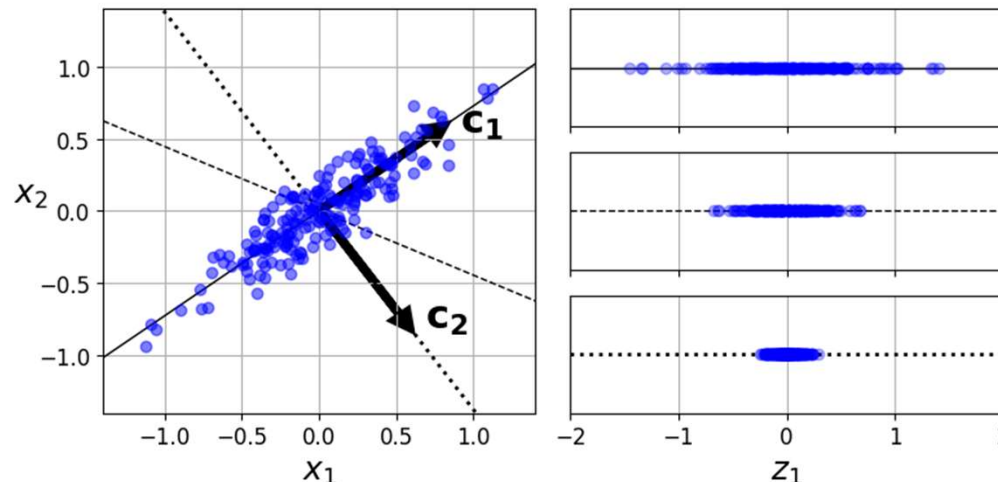
Example of SVD

```
A = np.array([[3, 1, 4],  
              [1, 5, 9],  
              [2, 6, 5]])
```

- U: $\begin{bmatrix} -0.32463251 & -0.79898436 & -0.50619929 \\ -0.75307473 & -0.1054674 & 0.64942672 \\ -0.57226932 & 0.59203093 & -0.56745679 \end{bmatrix}$
- S: $[13.58235799 \ 2.84547726 \ 2.32869289]$
- V: $\begin{bmatrix} -0.21141476 & -0.55392606 & -0.80527617 \\ -0.46331722 & 0.78224635 & -0.41644663 \\ -0.86060499 & -0.28505536 & 0.42202191 \end{bmatrix}$

Principal component analysis (PCA)

- PCA is the most popular dimensionality reduction
- The first component of PCA is called the principal component
- The first component accounts for the largest variance in data
- With each subsequent component, less information is contributed



Projecting down to d dimensions

- We can reduce the dimensionality to d by projecting the original data onto the hyperplane by the first d principal components:

$$\mathbf{X}_{d(\text{projected})} = \mathbf{X}\mathbf{W}_d$$

- $\mathbf{X}_{d(\text{projected})}$: Reduced dataset
- $\mathbf{X}$: Original data in a higher-dimensional space
- $\mathbf{W}_d$: Matrix containing the first d columns of the eigenvectors

Lost Information

- Explained ratio variance shows the proportion of variance captured by each principal component

```
A = np.array([[3, 1, 4],  
              [1, 5, 9],  
              [2, 6, 5]])
```

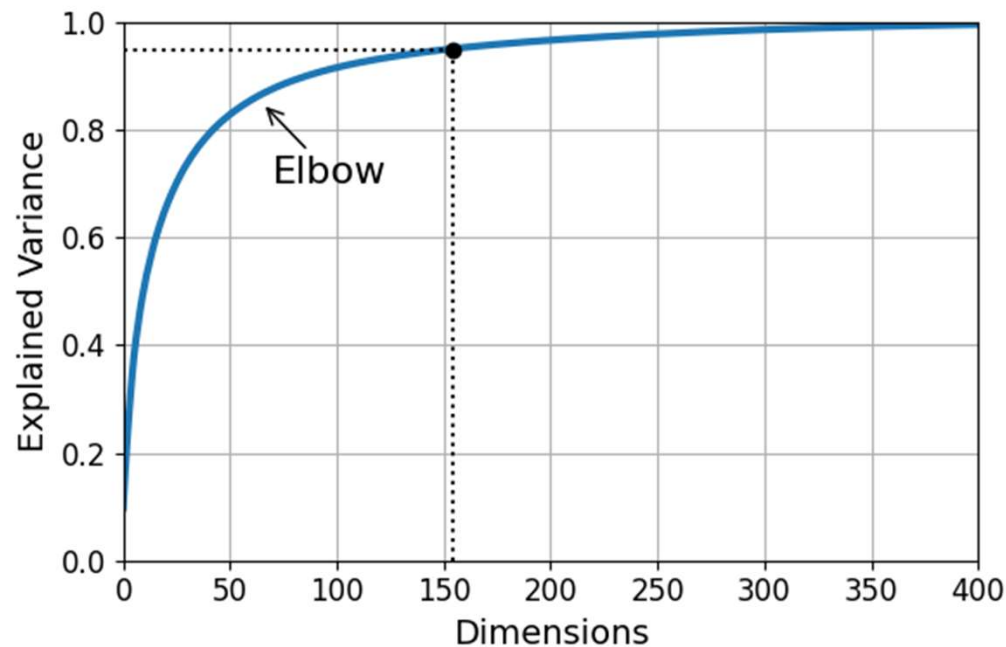
- U: $\begin{bmatrix} -0.32463251 & -0.79898436 & -0.50619929 \\ -0.75307473 & -0.1054674 & 0.64942672 \\ -0.57226932 & 0.59203093 & -0.56745679 \end{bmatrix}$
- S: $[13.58235799 \ 2.84547726 \ 2.32869289]$
- V: $\begin{bmatrix} -0.21141476 & -0.55392606 & -0.80527617 \\ -0.46331722 & 0.78224635 & -0.41644663 \\ -0.86060499 & -0.28505536 & 0.42202191 \end{bmatrix}$

- `pca.explained_variance_ratio_`
 $[0.93171944 \ 0.04089263 \ 0.02738793]$

- 2.73% of information is lost when we use the first two eigenvectors

The right number of d (lower dimension)

- Plot the explained variance versus the number of dimensions
- Pick the elbow (slope= 45 degrees)
- The following plot is based on SVD of MNIST data



Is it possible to recover the lost information

- Some information is lost when we project the data

$$\mathbf{X}_{d(\text{projected})} = \mathbf{X}\mathbf{W}_d$$

- Is it possible to recover the lost information by reversing the projection?

$$\mathbf{X}_{\text{recovered}} = \mathbf{X}_{d(\text{projected})}\mathbf{W}_d^T$$

- Let us check the next slide

Determine whether we fully recovered the samples

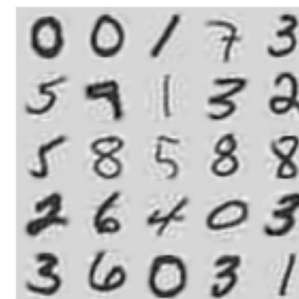
- We applied PCA with a different number of components (d) but applied the reverse projection



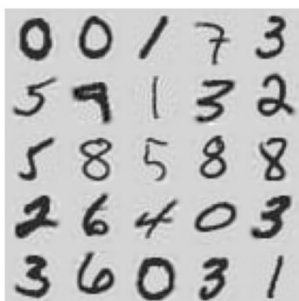
$d=25$



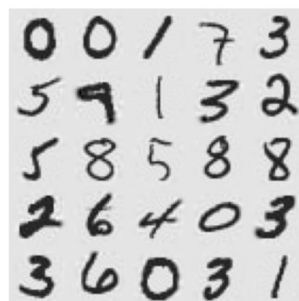
$d=50$



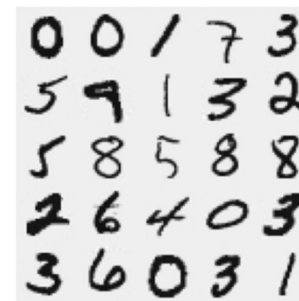
$d=100$



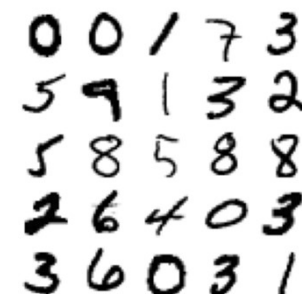
$d=200$



$d=300$



$d=500$

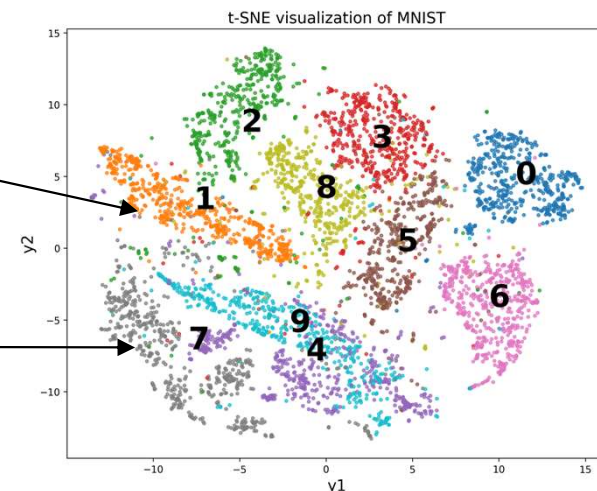
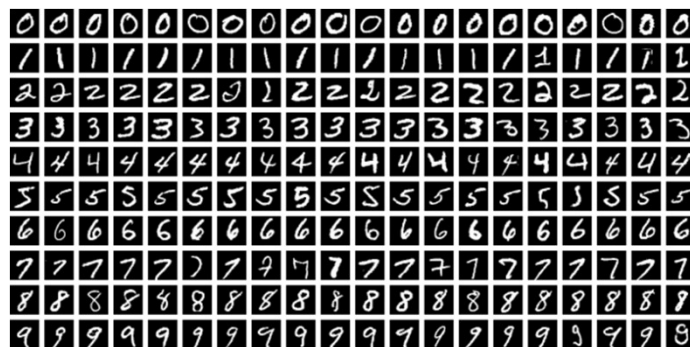


Original
($n=784$)

Manifold learning

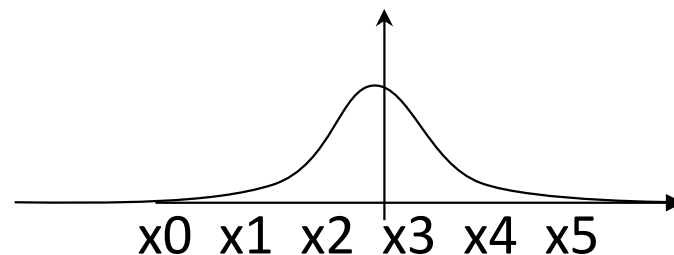
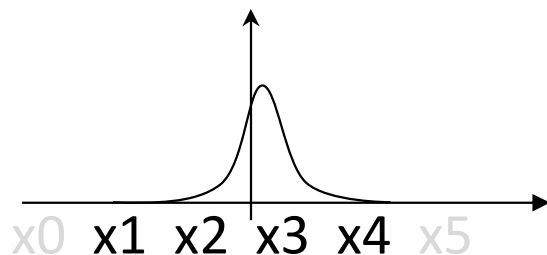
Two methods of manifold learning

- First: Stochastic Neighbor Embedding (SNE)
- Second: t-Distributed Stochastic Neighbor Embedding (t-SNE)
- Both help interpret the underlying features of unlabeled data (unsupervised)
- Both are useful in handling high-dimensional data because represent them in 2d



Stochastic Neighbor Embedding (SNE)

1. Converts the distance in a high-dimensional space into probabilities: It finds a Gaussian probability ($p_{i,j}$) of two points (x_i, x_j) being neighbors. Closer points are more likely to be neighbors.
2. A wide probability picks distant points (x_0, x_5) as neighbors



3. It also finds the probability ($q_{i,j}$) of the projected points (y_i, y_j) in a low-dimensional space

Main idea of SNE

- SNE minimizes the cost by reducing the difference between $p_{i,j}$ and $q_{i,j}$ using KL (Kullback-Leibler) divergence:

$$\text{cost} = \sum_i \sum_j p_{i|j} \log \frac{p_{i|j}}{q_{i|j}}$$

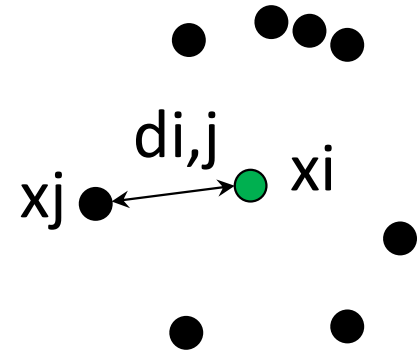
- We want neighbors (x_i, x_j) in a high-dimensional space to remain neighbors (y_i, y_j) in a low-dimensional space
- We tune the parameters of the probabilities ($p_{i,j}$ and $q_{i,j}$)

Key properties of SNE

- SNE uses gradient descent to minimize the cost
- SNE asks us to specify the number of neighbors
- The number of neighbors is called **perplexity** in SNE (it is usually between 5 and 50)
- We should search over the tuning parameters to find the best answer (discussed later in this lecture)

Crowding problem in SNE

- Crowding problem happens when some points are equally distanced ($d_{i,j}$) from a specific point (x_i)
- These equidistant points overwhelm SNE
- SNE struggles if we try to space neighboring points in a low-dimensional space
- t-SNE addresses the crowding problem (next slide)



t-Distributed Stochastic Neighbor Embedding (t-SNE)

- t-SNE addresses the crowding problem by using Student's t-distribution (Cauchy distribution) in a low-dimensional space
- The only difference between t-SNE and SNE is the probability distribution in a low-dimensional space
- Often, t-SNE outperforms in practice, but we can check both methods to find the best results

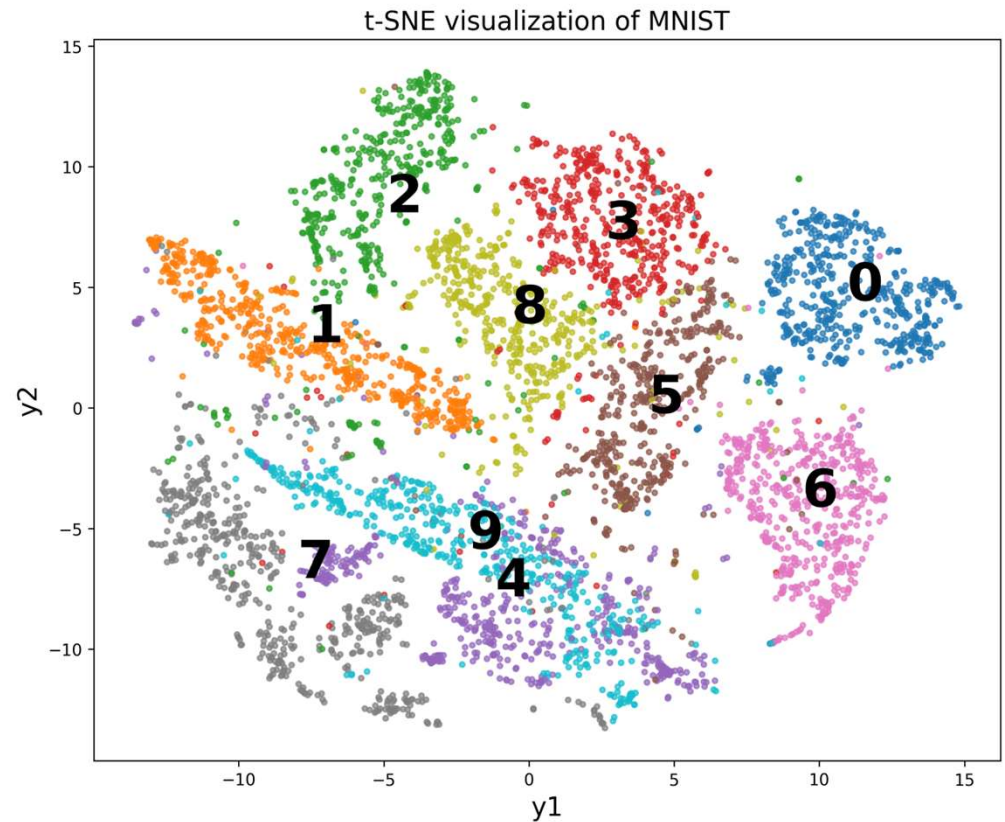
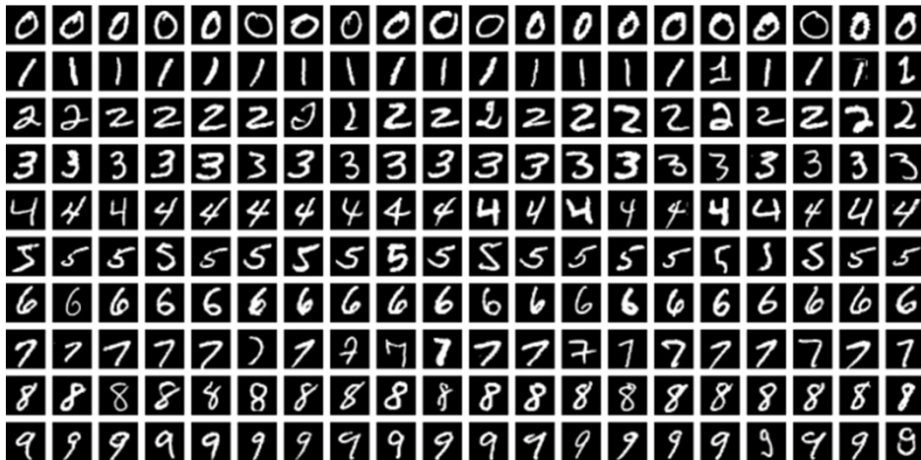
Implementation of t-SNE

```
tsne = TSNE(n_components=2, perplexity=30, n_iter=300,  
            learning_rate=200)
```

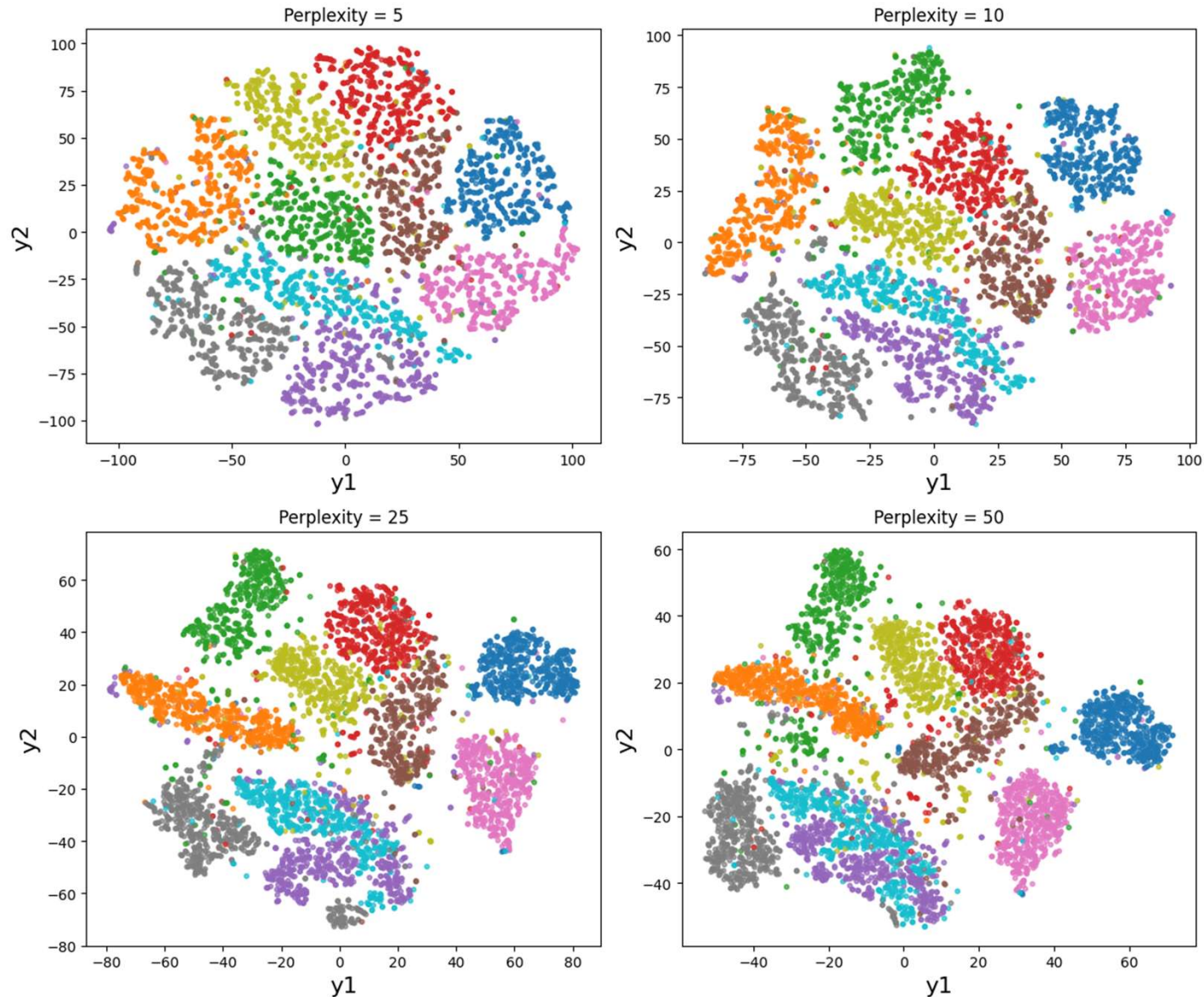
- `n_components` shows the number of lower dimensions (2 or 3 is preferred for visualization)
- Perplexity is the number of neighbors, as discussed earlier
- `n_iter` is the maximum number of iterations
- `learning_rate` controls the step size (remember that T-SNE uses the gradient descent method)

Analyzing 5000 handwritten digits of MNIST using t-SNE

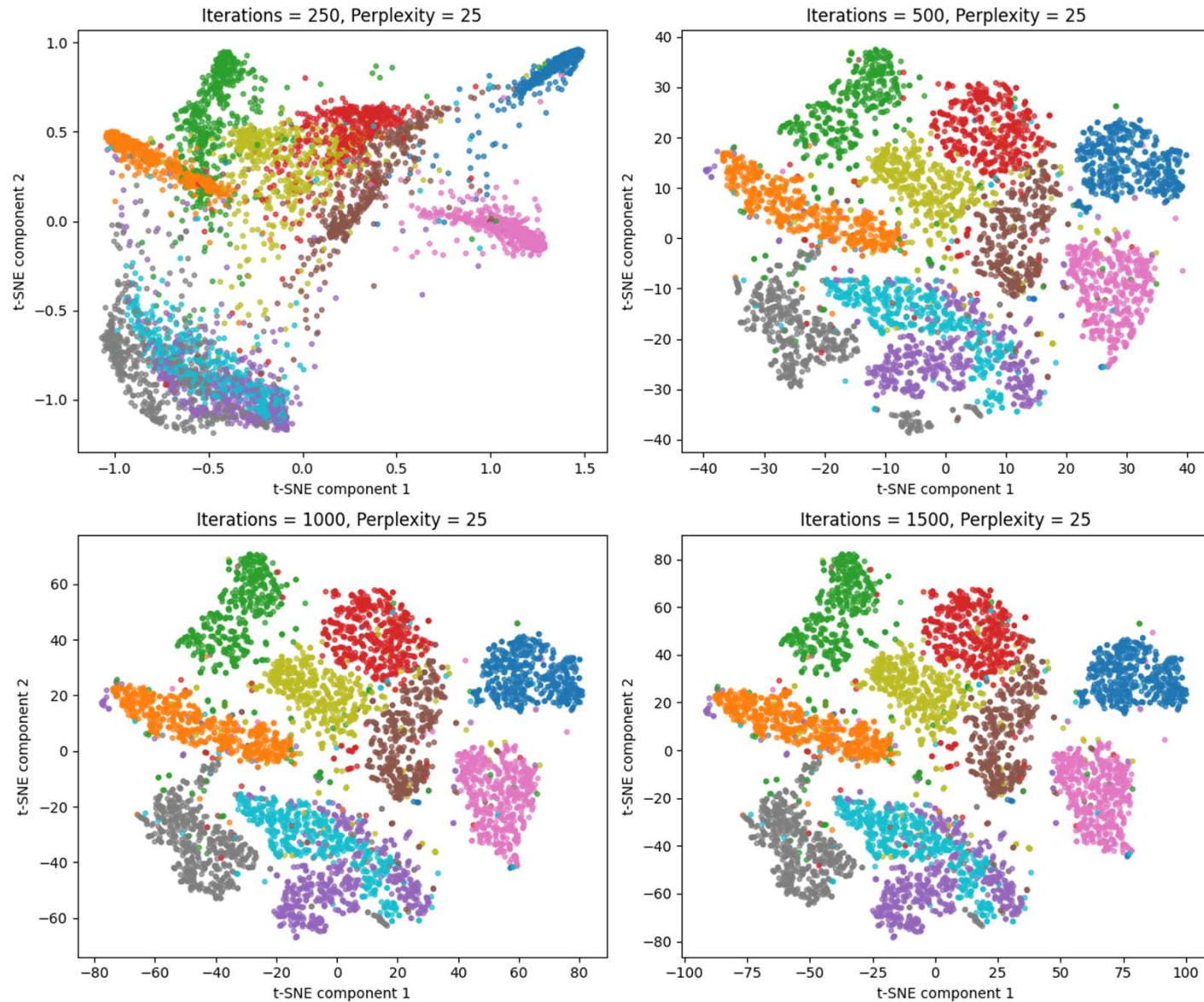
- tsne = TSNE(n_components=2, perplexity=30, n_iter=300, learning_rate=200)



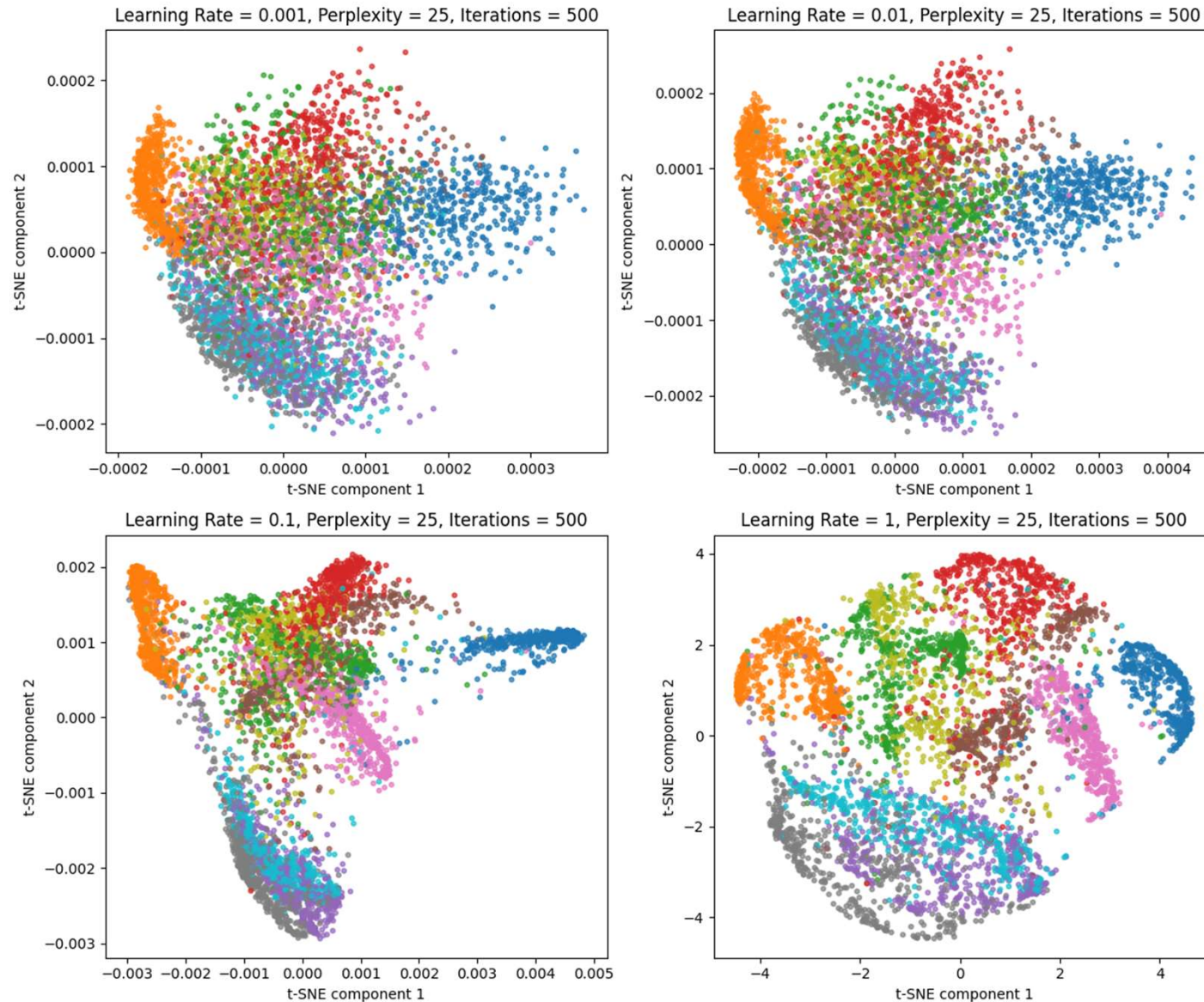
Effects of perplexity on t-SNE (choose the best performance)



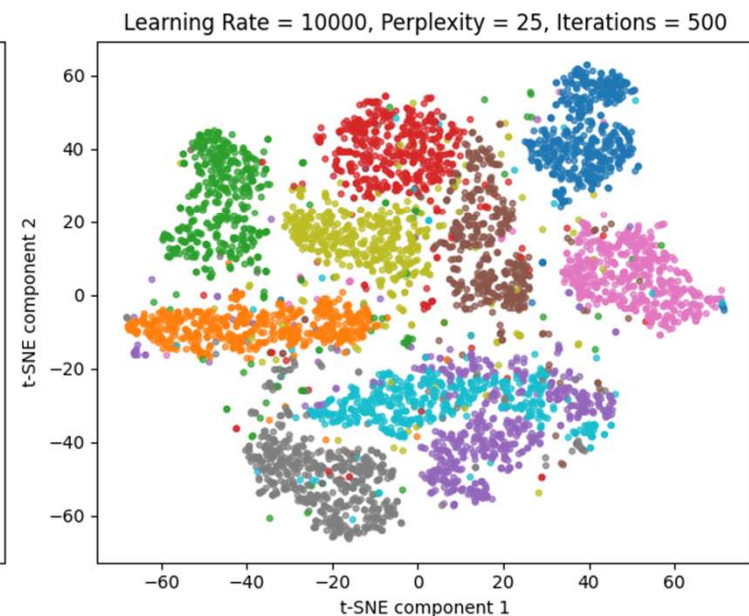
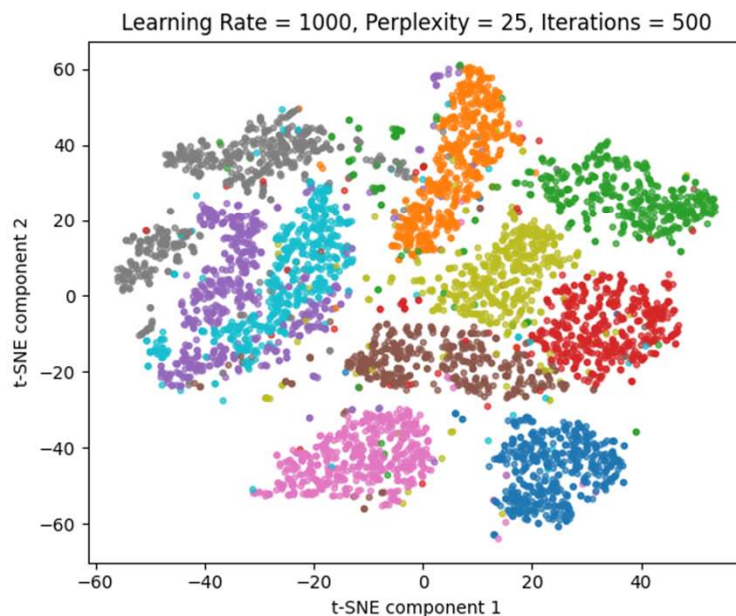
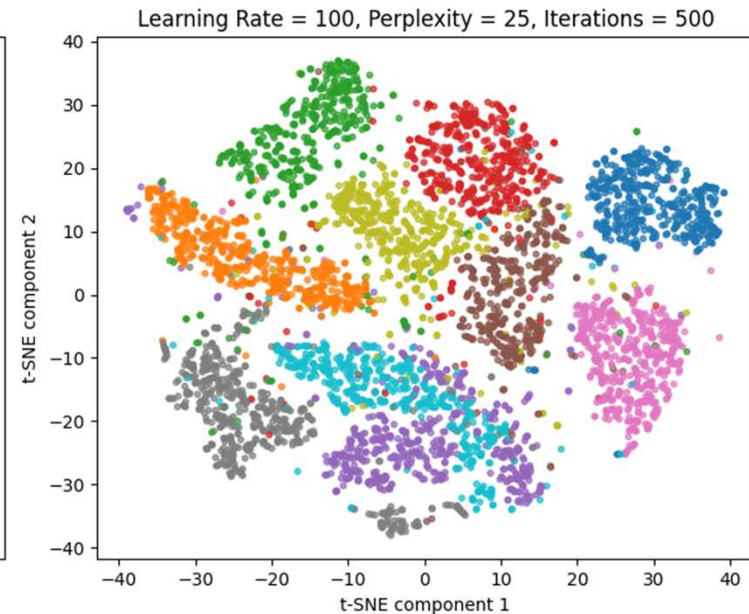
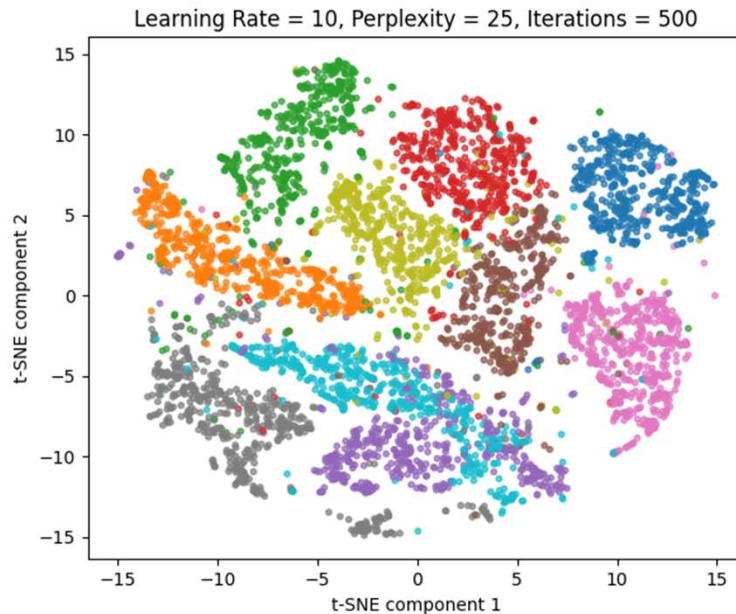
Effects of number of iterations on t-SNE (perplexity=25)



Effects of learning rate on t-SNE (perplexity=25, num_iteratations=500)



Effects of learning rate on t-SNE (continued)



Other methods of projection and manifold learning

- Random projection: briefly discussed next
- Locally linear embedding (LLE): briefly discussed next
- Multidimensional Scaling (MDS): check [manifold.MDS](#) in scikit-learn
- Isomap Embedding: check [manifold.Isomap](#) in scikit-learn
- Linear discriminant analysis (LDA): check `LinearDiscriminantAnalysis` in scikit-learn

Random projection

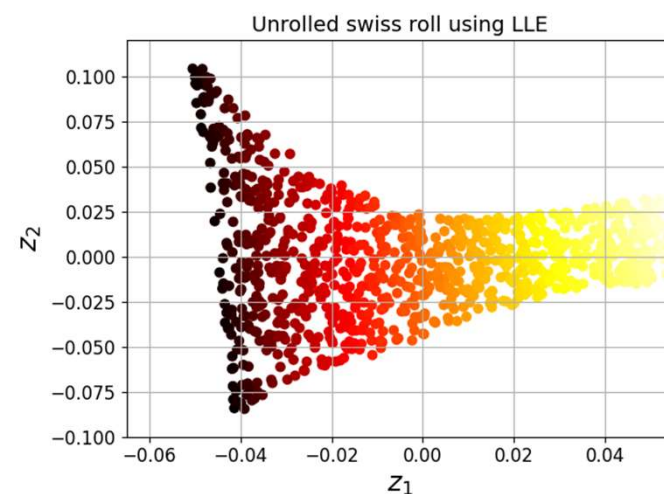
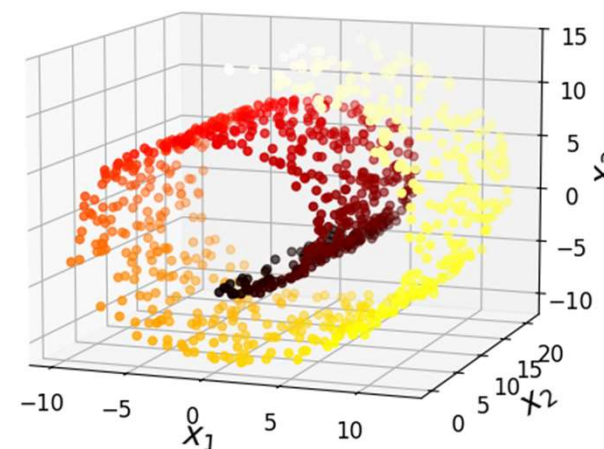
- It projects data using a **random** linear operation
- It uses a matrix P filled by a Gaussian distribution (mean=0, variance=1)
- Shape of $P=[d,n]$
- d : number of target dimensions

$$d \geq 4 \log(m) / \left(\frac{\varepsilon^2}{2} - \frac{\varepsilon^3}{3} \right)$$

- n : number of original dimensions
- m : number of samples (instances)
- ε : tuning parameter (usually smaller than 10%)

Locally linear embedding (LLE)

- This is a nonlinear method
- It is a manifold learning technique
- It works well when noise is limited
- It finds the k-nearest neighbors of the data
- It does not preserve the global distances



Popup questions

- PCA is used for feature selection (True or False)
- PCA is used for visualization (True or False)
- T-SNE is used for feature engineering (True or False)
- T-SNE is used for feature selection (True or False)